

Learning-Guided Higher-Order Automated Reasoning for Isabelle/HOL

Jan Jakubův¹ Cezary Kaliszyk² Martin Suda¹

AITP 2026, Aussois, France, 1st September, 2026



DEEPER project · supported by Renaissance Philanthropy
and the Czech Science Foundation, project no. 24-12759S

¹Czech Technical University in Prague, Czech Republic

²University of Melbourne, Australia

Introduction

Isabelle/HOL Export and Benchmark

Higher-Order ENIGMA and Deepire

Evaluation

Summary and Future Work

- Interactive theorem provers (ITPs) such as Isabelle/HOL combine expressive logics with large formal libraries.

- Interactive theorem provers (ITPs) such as **Isabelle/HOL** combine expressive logics with large formal libraries.
- **Hammer systems** (e.g. Sledgehammer) bridge ITPs and external automatic theorem provers (ATPs):
 - Select relevant facts from the library (premise selection).
 - Export the proof obligation to an ATP format.
 - Reconstruct successful ATP proofs inside the ITP.

- Interactive theorem provers (ITPs) such as **Isabelle/HOL** combine expressive logics with large formal libraries.
- **Hammer systems** (e.g. Sledgehammer) bridge ITPs and external automatic theorem provers (ATPs):
 - Select relevant facts from the library (premise selection).
 - Export the proof obligation to an ATP format.
 - Reconstruct successful ATP proofs inside the ITP.
- **Two complementary learning problems:**
 - **Premise selection:** which facts to include?
 - **Clause guidance:** which clauses to explore inside the ATP?

- Interactive theorem provers (ITPs) such as **Isabelle/HOL** combine expressive logics with large formal libraries.
- **Hammer systems** (e.g. Sledgehammer) bridge ITPs and external automatic theorem provers (ATPs):
 - Select relevant facts from the library (premise selection).
 - Export the proof obligation to an ATP format.
 - Reconstruct successful ATP proofs inside the ITP.
- **Two complementary learning problems:**
 - **Premise selection:** which facts to include?
 - **Clause guidance:** which clauses to explore inside the ATP?
- Both problems have been successfully tackled in the **first-order** (FO) setting.

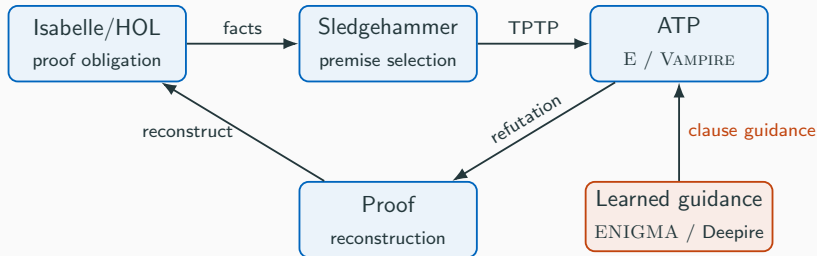
- **ENIGMA**: learning-guided clause selection for E.
 - Gradient-boosted decision trees classify clauses as useful/useless.
 - Positive = clauses appearing in a proof; negative = others.

- **ENIGMA**: learning-guided clause selection for E.
 - Gradient-boosted decision trees classify clauses as useful/useless.
 - Positive = clauses appearing in a proof; negative = others.
- **Deepire 2.0**: neural clause selection guidance for VAMPIRE.
 - Graph Neural Network (GNN) embeds the problem.
 - Recursive Neural Networks (RvNN) score derived clauses.

- **ENIGMA**: learning-guided clause selection for E.
 - Gradient-boosted decision trees classify clauses as useful/useless.
 - Positive = clauses appearing in a proof; negative = others.
- **Deepire 2.0**: neural clause selection guidance for VAMPIRE.
 - Graph Neural Network (GNN) embeds the problem.
 - Recursive Neural Networks (RvNN) score derived clauses.
- Both are trained on large benchmarks and run inside the ATP's saturation loop.

- **ENIGMA**: learning-guided clause selection for E.
 - Gradient-boosted decision trees classify clauses as useful/useless.
 - Positive = clauses appearing in a proof; negative = others.
- **Deepire 2.0**: neural clause selection guidance for VAMPIRE.
 - Graph Neural Network (GNN) embeds the problem.
 - Recursive Neural Networks (RvNN) score derived clauses.
- Both are trained on large benchmarks and run inside the ATP's saturation loop.
- **This work**: extend both to the **higher-order** (HO) setting using Isabelle/HOL exports.

The Hammer Pipeline



- Both **ENIGMA** and **Deepire 2.0** plug into the ATP's given-clause loop, guiding which clause to explore next.
- **This work**: make this loop work natively on **HO** problems.

Introduction

Isabelle/HOL Export and Benchmark

Higher-Order ENIGMA and Deepire

Evaluation

Summary and Future Work

Exporting Isabelle/HOL Proof Obligations

- We use **Mirabelle** to replay Isabelle/HOL developments and collect intermediate proof obligations.

Exporting Isabelle/HOL Proof Obligations

- We use **Mirabelle** to replay Isabelle/HOL developments and collect intermediate proof obligations.
- Sledgehammer exports each obligation in TPTP format:
 - **TF0**: monomorphic first-order. $[\text{atom}_0 \rightarrow \text{atom}_1 \rightarrow \text{atom}_2]$
 - **TH0**: monomorphic higher-order. $[(i \rightarrow i) \rightarrow \text{list}_i \rightarrow \text{list}_i]$
 - **TH1**: *polymorphic* higher-order. $[\forall \alpha. (\alpha \rightarrow \alpha) \rightarrow \text{list } \alpha \rightarrow \text{list } \alpha]$

Exporting Isabelle/HOL Proof Obligations

- We use **Mirabelle** to replay Isabelle/HOL developments and collect intermediate proof obligations.
- Sledgehammer exports each obligation in TPTP format:
 - **TF0**: monomorphic first-order. $[\text{atom}_0 \rightarrow \text{atom}_1 \rightarrow \text{atom}_2]$
 - **TH0**: monomorphic higher-order. $[(i \rightarrow i) \rightarrow \text{list}_i \rightarrow \text{list}_i]$
 - **TH1**: *polymorphic* higher-order. $[\forall \alpha. (\alpha \rightarrow \alpha) \rightarrow \text{list } \alpha \rightarrow \text{list } \alpha]$
- Based on the stable **Isabelle-2025-2** release.

Exporting Isabelle/HOL Proof Obligations

- We use **Mirabelle** to replay Isabelle/HOL developments and collect intermediate proof obligations.
- Sledgehammer exports each obligation in TPTP format:
 - **TF0**: monomorphic first-order. $[\text{atom}_0 \rightarrow \text{atom}_1 \rightarrow \text{atom}_2]$
 - **TH0**: monomorphic higher-order. $[(i \rightarrow i) \rightarrow \text{list}_i \rightarrow \text{list}_i]$
 - **TH1**: *polymorphic* higher-order. $[\forall \alpha. (\alpha \rightarrow \alpha) \rightarrow \text{list } \alpha \rightarrow \text{list } \alpha]$
- Based on the stable **Isabelle-2025-2** release.
- MeSh premise filter; 512 facts per slice; no ITE/let/choice.

Exporting Isabelle/HOL Proof Obligations

- We use **Mirabelle** to replay Isabelle/HOL developments and collect intermediate proof obligations.
- Sledgehammer exports each obligation in TPTP format:
 - **TF0**: monomorphic first-order. $[\text{atom}_0 \rightarrow \text{atom}_1 \rightarrow \text{atom}_2]$
 - **TH0**: monomorphic higher-order. $[(i \rightarrow i) \rightarrow \text{list}_i \rightarrow \text{list}_i]$
 - **TH1**: *polymorphic* higher-order. $[\forall \alpha. (\alpha \rightarrow \alpha) \rightarrow \text{list } \alpha \rightarrow \text{list } \alpha]$
- Based on the stable **Isabelle-2025-2** release.
- MeSh premise filter; 512 facts per slice; no ITE/let/choice.
- Total: 246,277 problems per dialect.

Running the export at scale revealed several issues:

- **Name clash in η -expansion**: TPTP generator produced type-invalid problems; affected many tasks in HOL-Analysis.

Running the export at scale revealed several issues:

- **Name clash in η -expansion**: TPTP generator produced type-invalid problems; affected many tasks in HOL-Analysis.
- **HOL-Nominal Class1**: clashes between names and conames.

Running the export at scale revealed several issues:

- **Name clash in η -expansion**: TPTP generator produced type-invalid problems; affected many tasks in HOL-Analysis.
- **HOL-Nominal Class1**: clashes between names and conames.
- Smaller monomorphisation issues in lower-logic encodings.

Running the export at scale revealed several issues:

- **Name clash in η -expansion**: TPTP generator produced type-invalid problems; affected many tasks in HOL-Analysis.
- **HOL-Nominal Class1**: clashes between names and conames.
- Smaller monomorphisation issues in lower-logic encodings.
- **All fixed** in the Isabelle development version.
- These fixes benefit *all* provers, not just ours.

Introduction

Isabelle/HOL Export and Benchmark

Higher-Order ENIGMA and Deepire

Evaluation

Summary and Future Work

Both E and VAMPIRE use a **given-clause saturation loop**:

1. Select the best *given clause* from the unprocessed/passive set.
2. Apply inference rules to derive new clauses.
3. Repeat until refutation or saturation.

Both E and VAMPIRE use a **given-clause saturation loop**:

1. Select the best *given clause* from the unprocessed/passive set.
 2. Apply inference rules to derive new clauses.
 3. Repeat until refutation or saturation.
- **λE** ($E \geq 3.0$): HO extension supporting monomorphic TH0.
 - **VAMPIRE**: Supports TH0 and also polymorphic TH1.

Higher-Order Saturation Provers

Both E and VAMPIRE use a **given-clause saturation loop**:

1. Select the best *given clause* from the unprocessed/passive set.
 2. Apply inference rules to derive new clauses.
 3. Repeat until refutation or saturation.
- **λE** ($E \geq 3.0$): HO extension supporting monomorphic TH0.
 - **VAMPIRE**: Supports TH0 and also polymorphic TH1.
 - Clause selection is the key decision point — target for learning.

λ ENIGMA: HO Features for ENIGMA

- In λE , HO (partial) applications $(F X Y)$ are stored as $@(F, X, Y)$ using an internal symbol $@$.
 λ -abstractions $(\lambda x.T)$ become $\lambda(x, T)$.

λ ENIGMA: HO Features for ENIGMA

- In λE , HO (partial) applications $(F X Y)$ are stored as $@(F, X, Y)$ using an internal symbol $@$.
 λ -abstractions $(\lambda x. T)$ become $\lambda(x, T)$.
- ENIGMA extracts **vertical** (top-down paths) and **horizontal** (head + argument heads) features from clause terms.

λ ENIGMA: HO Features for ENIGMA

- In λE , HO (partial) applications $(F\ X\ Y)$ are stored as $@(F, X, Y)$ using an internal symbol $@$.
 λ -abstractions $(\lambda x.T)$ become $\lambda(x, T)$.
- ENIGMA extracts **vertical** (top-down paths) and **horizontal** (head + argument heads) features from clause terms.
- λ ENIGMA extensions:
 - $@(F, X, Y)$: X, Y treated as children of F , not siblings: $F(X, Y)$.
 - $\lambda(x, T)$: variable name dropped; binder position marked.
 - Types embedded into features: arrow type $(i \rightarrow (i \rightarrow i) \rightarrow o)$ encoded as $i-(i-i)-o$ and appended to symbol names.

λ ENIGMA: HO Features for ENIGMA

- In λE , HO (partial) applications $(F\ X\ Y)$ are stored as $@(F, X, Y)$ using an internal symbol $@$.
 λ -abstractions $(\lambda x. T)$ become $\lambda(x, T)$.
- ENIGMA extracts **vertical** (top-down paths) and **horizontal** (head + argument heads) features from clause terms.
- λ ENIGMA extensions:
 - $@(F, X, Y)$: X, Y treated as children of F , not siblings: $F(X, Y)$.
 - $\lambda(x, T)$: variable name dropped; binder position marked.
 - Types embedded into features: arrow type $(i \rightarrow (i \rightarrow i) \rightarrow o)$ encoded as $i-(i-i)-o$ and appended to symbol names.
- Works on **TF0** and **TH0**.

- Deepire 2.0 architecture: GNN + RvNN (term structure) + RvNN (derivation history) + fully connected scoring network.

- Deepire 2.0 architecture: GNN + RvNN (term structure) + RvNN (derivation history) + fully connected scoring network.
- Adapted to handle **polymorphic HO problems**:
 - New GNN node type for **type constructors**.
 - Sorts promoted to full expressions (with sort variables).
 - Distinct node labels for universal syntactic concepts: arrow type, apply, λ , de Bruijn indices, propositional connectives.

- Deepire 2.0 architecture: GNN + RvNN (term structure) + RvNN (derivation history) + fully connected scoring network.
- Adapted to handle **polymorphic HO problems**:
 - New GNN node type for **type constructors**.
 - Sorts promoted to full expressions (with sort variables).
 - Distinct node labels for universal syntactic concepts: arrow type, apply, λ , de Bruijn indices, propositional connectives.
- Works on **TF0**, **TH0**, and **TH1**.

Introduction

Isabelle/HOL Export and Benchmark

Higher-Order ENIGMA and Deepire

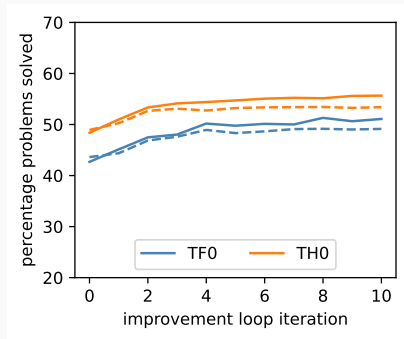
Evaluation

Summary and Future Work

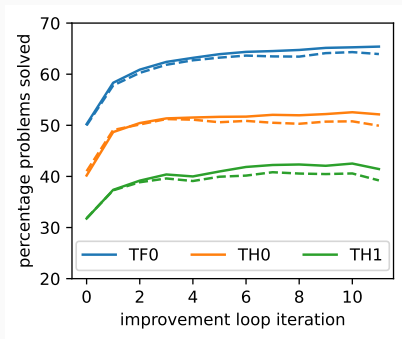
- 120,000 training problems; 5,000 test problems (aligned across dialects).
- **λ ENIGMA:**
 - 1,000-problem development set for model selection.
 - 10 training iterations; ~ 5 days per dialect.
 - 10 s per problem.
 - Evaluated on TF0 and TH0.
- **Deepire-2.0-HO:**
 - 10 training iterations; ~ 4.5 days per dialect.
 - 16 billion CPU instructions (~ 6 s).
 - Evaluated on TF0, TH0, and TH1.

Training Plots: λ ENIGMA and Deepire-2.0-HO

left = λ ENIGMA



right = Deepire-2.0-HO



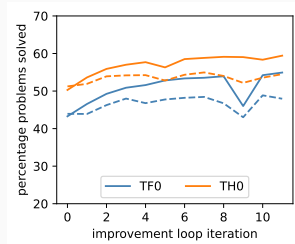
Solid = train, dashed = test. X-axis: training iteration (0 = baseline).

Both systems now trained on 120,000 problems.

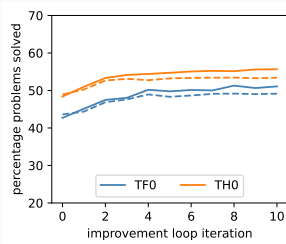
Impact of Training Set Size (λ ENIGMA)

Since our last talk, λ ENIGMA training was scaled up $4\times$:

29,000 problems (previous)



120,000 problems (now)



- Relative test-set gains over baseline:
 - TF0: 11.2 % (29K) $\rightarrow$ 12.66 % (120K).
 - TH0: 7.26 % (29K) $\rightarrow$ 9.12 % (120K).
- Both fragments keep improving; TF0 benefits somewhat more.
- Motivates scaling further to the full 246,277 problems.

- λ E baseline on TH0 outperforms TF0 — and even VAMPIRE.
- λ ENIGMA improves consistently on both fragments (test set):
 - +12.66 % on TF0 (from 43.6 % to 49.1 %).
 - +9.12 % on TH0 (from 48.9 % to 53.4 %).

Results: Deepire-2.0-HO

- VAMPIRE + Deepire 2.0 on TF0 is still the strongest single dialect: $\sim 65\%$ test coverage.
- Deepire-2.0-HO brings comparable relative gains on HO fragments:
 - TH0: $40.2\% \rightarrow 51.2\%$ (+27 % relative).
 - TH1: $31.8\% \rightarrow 40.8\%$ (+28 % relative).
- HO starting points are lower than FO — understanding why is an open research question.

Complementarity of Strategies

Greedy cover of the best strategy per system and fragment, on the 5,000 test problems:

<i>System</i>	<i>Fragment</i>	<i>Solves</i>	<i>Total</i>	<i>Cumulative</i>
Deepire 2.0	TF0	3229	3229	64.58 %
λ ENIGMA	TH0	2670	3374	67.48 %
Deepire-2.0-HO	TH1	2041	3440	68.80 %
Deepire-2.0-HO	TH0	2561	3464	69.28 %
λ ENIGMA	TF0	2440	3480	69.60 %

- 3 HO strategies alone: 60.36 %
- 2 FO strategies alone: 65.38 %
- All five together: 69.60 %.
- HO strategies add +4.22 % (points) over FO alone.

Introduction

Isabelle/HOL Export and Benchmark

Higher-Order ENIGMA and Deepire

Evaluation

Summary and Future Work

Contributions:

- **λ ENIGMA**: first learning-guided HO clause guidance for λE (TH0 + TF0 from Isabelle).
- **Deepire-2.0-HO**: neural HO clause guidance for VAMPIRE (TF0 + TH0 + TH1).
- **Improved Sledgehammer export**: fixed bugs in HOL-Analysis and HOL-Nominal; improved monomorphisation.
- Consistent improvement over baselines; HO strategies are complementary to FO ones.

- Why does native TH1 solving currently lag behind TH0?
⇒ Investigate calculus, encoding, and strategy differences.
- Use HO premise selection to improve TH1 training data quality.
- Scale training to the full 246,277 problems.
- Closer integration between λ ENIGMA and Deepire-2.0-HO strategies.

- Why does native TH1 solving currently lag behind TH0?
⇒ Investigate calculus, encoding, and strategy differences.
- Use HO premise selection to improve TH1 training data quality.
- Scale training to the full 246,277 problems.
- Closer integration between λ ENIGMA and Deepire-2.0-HO strategies.

Thank you!